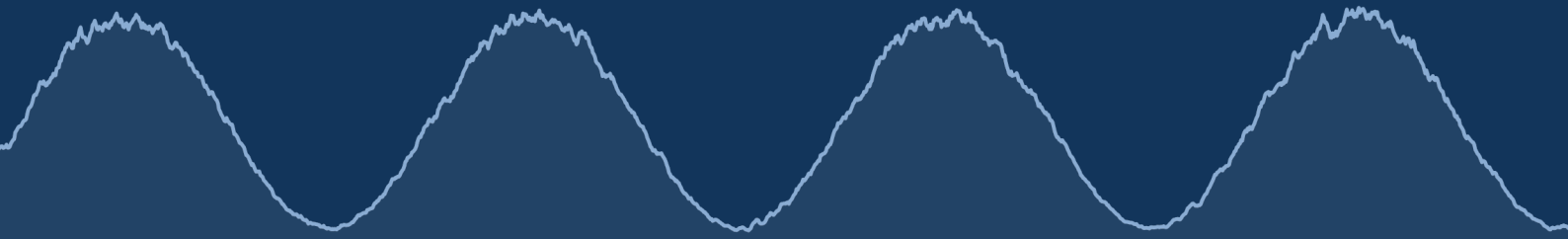
CSBB 422 · BIG DATA ANALYTICS · PROJECT REPORT

CoviSpark

Distributed COVID-19 data analysis with
Hadoop MapReduce, Apache Spark and a web dashboard



COURSE CSBB 422 — Big Data Analytics

Contents

1. Introduction 3

1.1 Objectives . . . 3

1.2 Scope . . . 3

2. Dataset 4

2.1 Sample generator . . . 4

3. System Architecture 5

4. Technologies Used 6

4.1 Hadoop configuration . . . 6

5. Implementation 7

5.1 MapReduce: country-wise totals . . . 7

5.2 Spark analysis jobs . . . 8

5.3 Dashboard . . . 9

6. Workflow and Execution 11

6.1 Cluster mode . . . 11

6.2 Local mode . . . 11

6.3 Web interfaces . . . 12

7. Results 12

Top five countries . . . 13

8. Testing and Verification 14

9. Limitations and Future Work 15

1. Introduction

CoviSpark is a course project for CSBB 422 (Big Data Analytics). It takes a COVID-19 dataset in the Our World in Data (OWID) format, stores it in HDFS, processes it with a Hadoop MapReduce job and a set of Apache Spark jobs, and shows the results on a small web dashboard. Everything runs in Docker on a single laptop, but the containers are arranged the way a real cluster would be: separate master and worker nodes for HDFS, YARN and Spark, talking to each other over a private network.

The point of the project is less the epidemiology and more the plumbing. We wanted to see one dataset go through the whole path that big-data coursework usually describes only on paper: distributed storage, a batch MapReduce job scheduled by YARN, an in-memory Spark pipeline, and finally a presentation layer that someone can actually look at.

1.1 Objectives

- Set up a multi-node Hadoop cluster (HDFS + YARN) and a Spark standalone cluster with Docker Compose.
- Load the COVID-19 dataset into HDFS with a replication factor of 2.
- Compute country-wise total cases and deaths with a Hadoop Streaming MapReduce job written in Python.
- Run five analytical Spark jobs: top countries, continent totals, a smoothed global infection curve, case-fatality rate and vaccination coverage.
- Serve the results through a Flask API and present them in a multi-view web dashboard.
- Keep a local mode that needs no Docker, so the analysis can be checked quickly on any machine.

1.2 Scope

The project works on a static CSV snapshot; there is no streaming ingestion. When the real OWID file is not available, a generator script produces a synthetic dataset with the same columns (36,000 rows for 25 countries). All numbers quoted in this report come from that synthetic dataset and should not be read as real pandemic statistics.

2. Dataset

The pipeline expects the column layout of `owid-covid-data.csv` from Our World in Data. Only eleven columns are used, so the same code runs on the full OWID file (which has around 67 columns) and on the generated sample.

Column	Type	Used by
iso_code	string	Filter: rows starting with OWID_ are continent/world aggregates and are dropped
continent	string	MapReduce key value, Spark Jobs 1, 2, 4, 5
location	string	Country name; the grouping key everywhere
date	date	Spark Job 3 (daily global series)
new_cases, new_deaths	number	MapReduce sums; Spark Jobs 2 and 3
total_cases, total_deaths	number	Spark Jobs 1 and 4 (maximum cumulative value)
people_vaccinated	number	Spark Job 5
total_vaccinations	number	Cast and cleaned, not used in a job
population	number	Spark Job 5 (denominator)

2.1 Sample generator

`data/generate_sample.py` builds the synthetic file. It uses a fixed random seed (42), so every run produces the same data. For each of 25 countries it walks day by day from 22 January 2020 to 31 December 2023 (1,440 days) and computes:

- new cases from a sine wave with a 60-day period, scaled by population and a random peak factor, with ±30% daily noise;
- new deaths as new cases times a per-country fatality rate between 0.5% and 2.5%;
- vaccinations starting after day 300 at about 0.1% of the population per day, capped at twice the population; `people_vaccinated` is 60% of total vaccinations.

Location names in the sample are ISO codes (IND, USA, BRA ...) rather than full country names. The scripts generate the file automatically if `data/owid-covid-data.csv` is missing; to use real data, download the OWID file to that path instead.

3. System Architecture

The cluster is defined in `docker-compose.yml` and consists of ten containers. Hadoop runs one NameNode, two DataNodes, one ResourceManager and two NodeManagers. Spark runs a master and two workers. The dashboard is a separate Flask container. Project folders on the host are bind-mounted into the containers, which is how scripts reach the cluster and how results come back out.

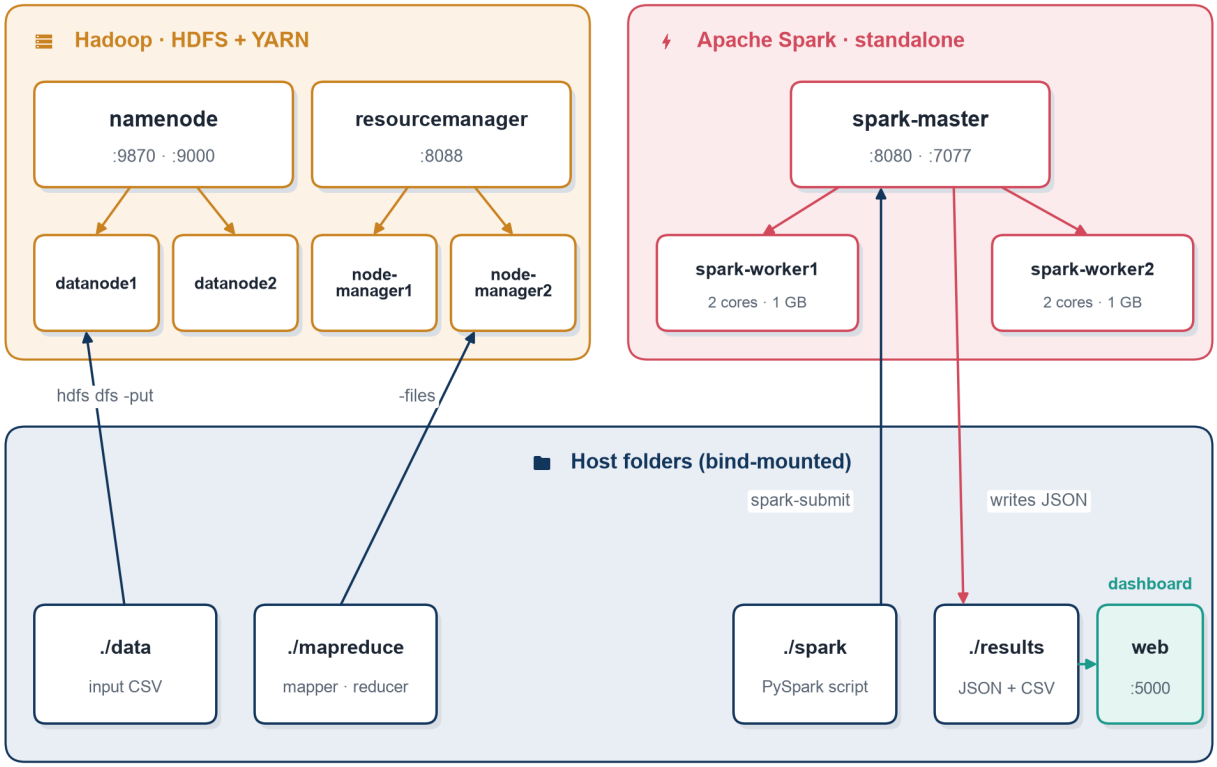


Figure 1. Containers, networks and shared folders

Container	Image	Role	Host port
namenode	bde2020/hadoop-namenode 3.2.1	HDFS metadata, job client	9870, 9000
datanode1, datanode2	bde2020/hadoop-datanode 3.2.1	HDFS block storage	—
resourcemanager	bde2020/hadoop-resource-manager 3.2.1	YARN scheduling	8088
nodemanager1, nodemanager2	bde2020/hadoop-nodemanager 3.2.1, plus python3 (local build)	Run map and reduce tasks	—
spark-master	bitnamilegacy/spark:3.5	Spark master, driver host	8080, 7077
spark-worker1, spark-worker2	bitnamilegacy/spark:3.5	Executors, 2 cores and 1 GB each	—
dashboard	local build on python:3.11-slim	Flask API and web page	5000

4. Technologies Used

Layer	Technology	Version	Why it is here
Storage	HDFS	3.2.1	Distributed, replicated input storage for MapReduce
Scheduling	YARN	3.2.1	Allocates containers for map and reduce tasks
Batch processing	Hadoop Streaming	3.2.1	Lets the mapper and reducer be plain Python scripts
Analytics	Apache Spark (PySpark)	3.5.6	DataFrame jobs, window functions, caching
Task runtime	Python	3.5 on NodeManagers, 3.11 dashboard	Mapper/reducer, Flask app
Web API	Flask	3.0.3	Serves the result files as JSON
Charts	Chart.js	4.4.1 (bundled)	Bar, doughnut and line charts in the browser
Orchestration	Docker Compose	Compose v2 or later	Starts the whole cluster with one command
Local mode	PySpark from pip, Flask, pandas	latest	Runs everything without Docker

4.1 Hadoop configuration

The XML files in `config/` are mounted into the NameNode, ResourceManager and both NodeManagers, so all nodes share one configuration. The settings that matter are:

File	Property	Value
core-site.xml	fs.defaultFS	hdfs://namenode:9000
hdfs-site.xml	dfs.replication	2 (one copy on each DataNode)
mapred-site.xml	mapreduce.framework.name	yarn
mapred-site.xml	map / reduce / AM memory	512 MB containers, -Xmx384m heaps
yarn-site.xml	yarn.resourcemanager.hostname	resourcemanager
yarn-site.xml	yarn.nodemanager.resource.memory-mb	1024 per NodeManager
yarn-site.xml	yarn.resourcemanager.bind-host	0.0.0.0 (UI reachable on localhost)
yarn-site.xml	yarn.nodemanager.vmem-check-enabled	false

The memory values are deliberately small. Docker Desktop on the test machine had 3.8 GB in total, and with default settings the ResourceManager and a NodeManager were killed by the kernel's out-of-memory handler partway through the job.

5. Implementation

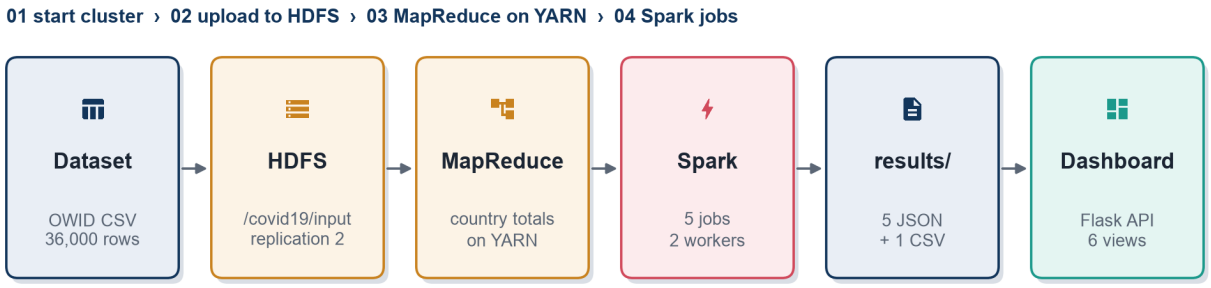


Figure 2. End-to-end data flow

5.1 MapReduce: country-wise totals

The MapReduce job is the simplest analysis and is meant to show the classic pattern. The mapper reads CSV lines, skips aggregate rows and emits one tab-separated record per row. Hadoop sorts the records by the first field, so all rows of one country reach the reducer together, and the reducer adds them up.

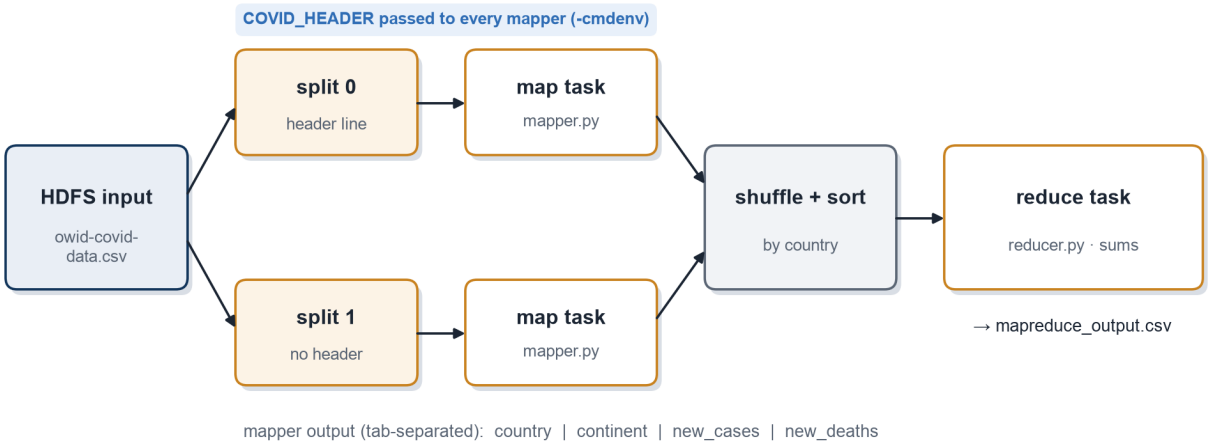


Figure 3. Hadoop Streaming job: splits, mappers, shuffle and reducer

One detail is easy to miss. HDFS splits the input file into blocks, and each mapper only sees its own split. Only the first split starts with the CSV header, so a mapper reading the second split cannot tell which column is which. The run script therefore reads the header once and passes it to every mapper as an environment variable:

```
HEADER="$(hdfs dfs -cat /covid19/input/owid-covid-data.csv | head -1)"
hadoop jar hadoop-streaming-3.2.1.jar \
  -files  /mapreduce/mapper.py,/mapreduce/reducer.py \
  -mapper "python3 mapper.py"  -reducer "python3 reducer.py" \
  -cmdenv "COVID_HEADER=$HEADER" \
  -input  /covid19/input/owid-covid-data.csv  -output /covid19/output/mapreduce/
```

In local pipe mode the variable is not set and the mapper falls back to treating the first line as the header. Both scripts avoid f-strings because the NodeManager image ships Python 3.5. Streaming also appends a tab to every output line (it treats the whole line as a key with an empty value), so

the export step strips it with `sed` before writing `results/mapreduce_output.csv`.

5.2 Spark analysis jobs

`spark/covid_analysis.py` loads the CSV once, casts the numeric columns to double, replaces missing values with zero, drops the OWID aggregate rows and caches the result. All five jobs then run against the cached DataFrame and each writes one JSON file.

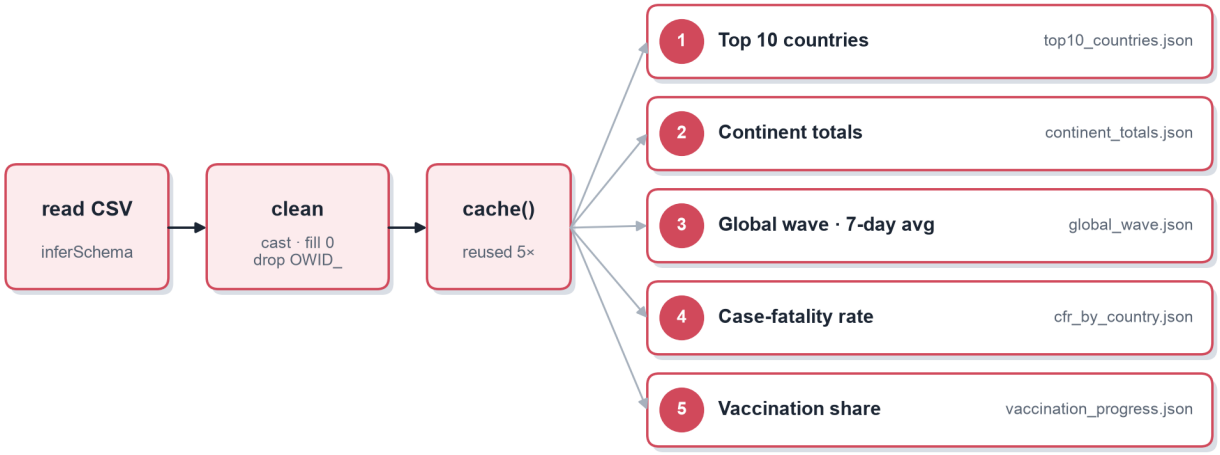


Figure 4. Spark pipeline: one cached DataFrame, five jobs

Job	What it computes	Spark operations	Output
1	Ten countries with the most cases	groupBy(location, continent), max(total_cases), orderBy desc, limit 10	top10_countries.json
2	Cases and deaths per continent	filter non-empty continent, groupBy, sum(new_cases), sum(new_deaths)	continent_totals.json
3	Global daily cases with 7-day centred average	groupBy(date), sum; avg over Window.orderBy(date).rowsBetween(-3, 3)	global_wave.json
4	Case-fatality rate, top 15	max totals per country, filter cases > 1000, deaths / cases × 100	cfr_by_country.json
5	Share of population vaccinated, top 15	max(people_vaccinated) / max(population) × 100	vaccination_progress.json

Job 3 is the only one that needs a window function. The window has no partitionBy clause, so Spark moves the 1,440 daily rows into a single partition to compute it. That is fine at this size; it is the first thing to change if the date range grows a lot.

```
win = Window.orderBy("date").rowsBetween(-3, 3)
daily_wave = (daily_global
    .withColumn("rolling_avg_cases",  F.avg("daily_cases").over(win))
    .withColumn("rolling_avg_deaths", F.avg("daily_deaths").over(win)))
```

5.3 Dashboard

The dashboard has two halves. `dashboard/app.py` is a small Flask application that does no analysis of its own: it reads the files in `results/` and returns them as JSON. If a file is missing it returns an empty list, so the page still loads before the jobs have run. The front end is plain HTML, CSS and JavaScript in `templates/` and `static/`, with Chart.js 4.4.1 bundled locally so nothing is fetched from the internet.

Endpoint	Source file	Notes
/	templates/index.html	Single-page dashboard
/api/status	all six outputs	Ready / missing, record count and modified time for each file
/api/summary	mapreduce_output.csv	Number of countries in the MapReduce output
/api/countries	mapreduce_output.csv	Every country with cases, deaths and CFR
/api/top10	top10_countries.json	Spark Job 1
/api/continents	continent_totals.json	Spark Job 2; also the source of global totals
/api/wave?step=N	global_wave.json	Spark Job 3; every Nth day (default 7, step=1 for all 1,440)
/api/cfr	cfr_by_country.json	Spark Job 4
/api/vaccination	vaccination_progress.json	Spark Job 5

The page is split into six views, reached from a sidebar:

View	What it shows
Overview	Five KPI cards (countries, cases, deaths, global CFR, peak 7-day average), the weekly infection curve, top 10 countries and a continent doughnut
Countries	All countries from the MapReduce output in a table that can be searched, filtered by continent and sorted by any column; CFR above the global rate is highlighted
Infection wave	The full daily series with year filters, toggles for the daily and averaged lines, and the peak, lowest and mean for the selected range
Case fatality	Top 15 CFRs with a dashed line at the global rate, plus a short explanation of the formula
Vaccination	Top 15 vaccination shares as bars and as progress rows with absolute numbers
Pipeline	The processing chain, a status table for every output file and links to the HDFS, YARN and Spark web interfaces

A light and a dark theme are available (the choice is remembered in the browser), the layout collapses to a single column with a slide-out menu on narrow screens, and an empty state explains how to run the pipeline when no results exist yet.

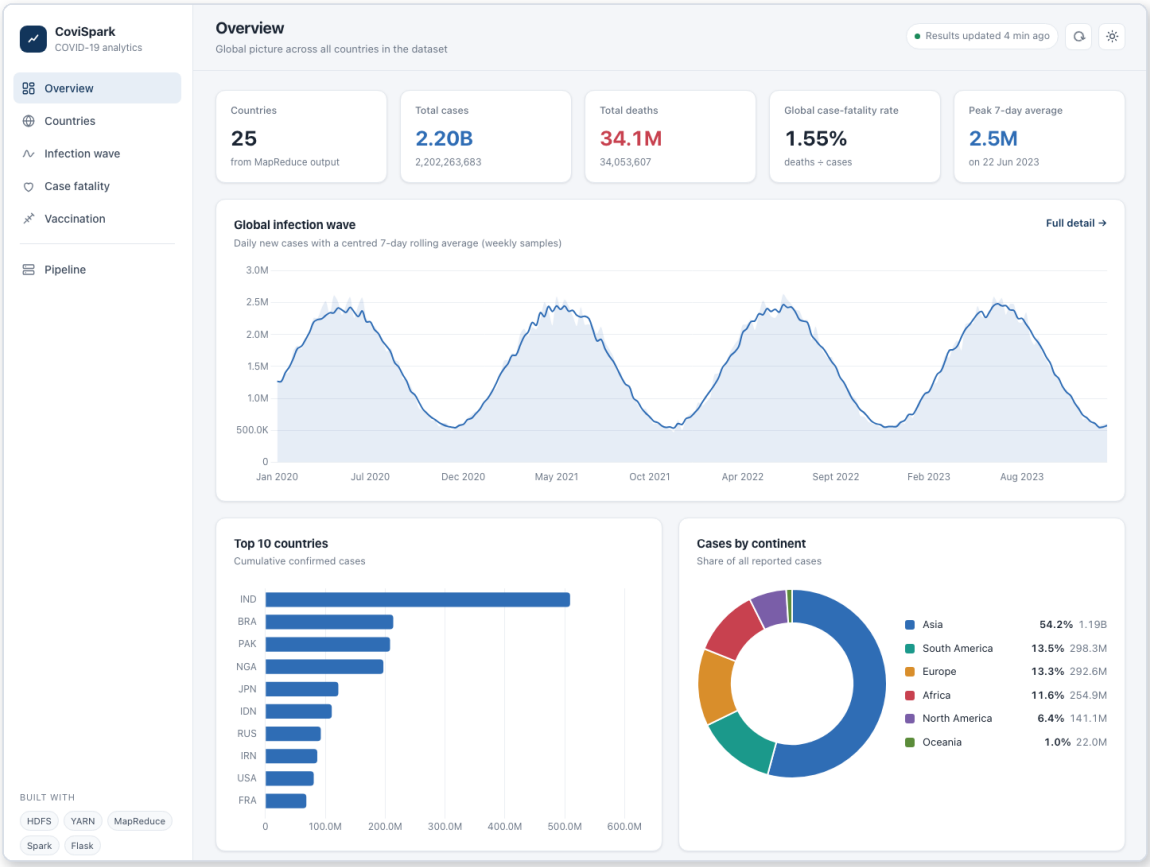
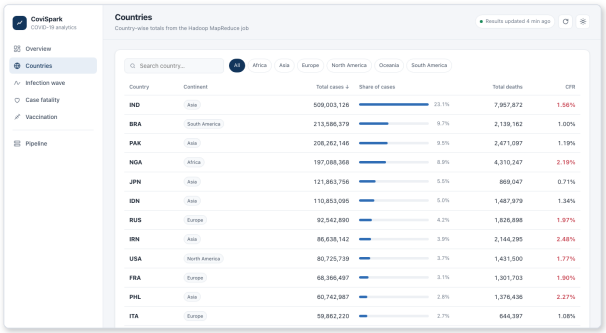
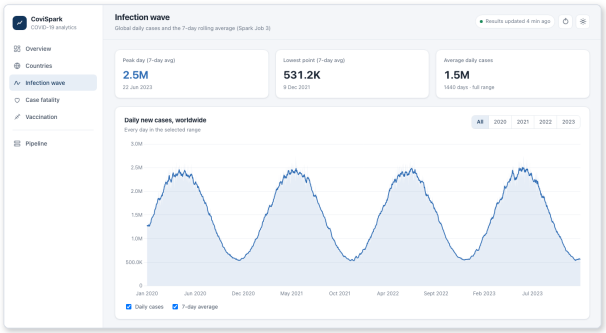


Figure 5. Overview view after a full cluster run

Countries



Infection wave



Case fatality



Pipeline

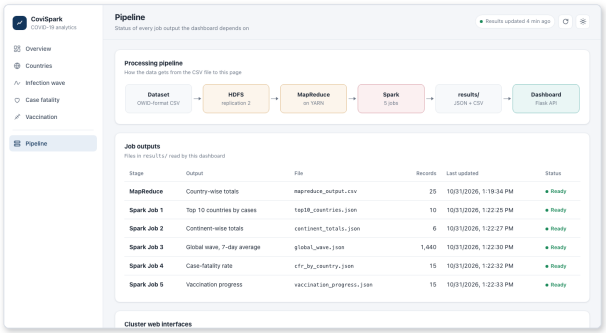


Figure 6. Countries, Infection wave, Case fatality and Pipeline views

6. Workflow and Execution

There are two ways to run the project. Cluster mode uses Docker and exercises HDFS, YARN and the Spark cluster. Local mode runs the same Spark script and the same mapper and reducer on the host, which is handy for checking changes quickly.

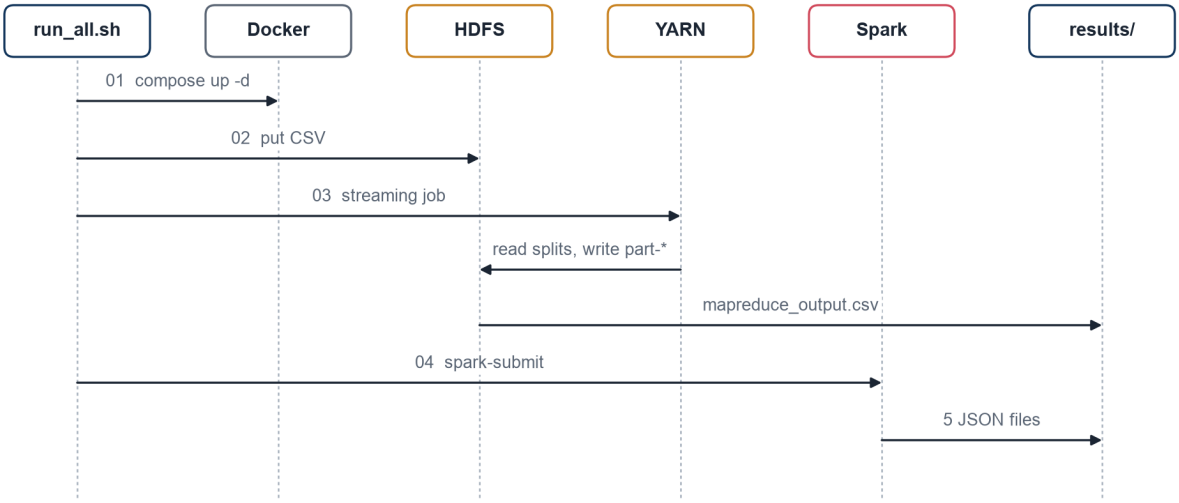


Figure 7. Order of operations in cluster mode (run_all.sh)

6.1 Cluster mode

```
bash run_all.sh        # runs scripts/01 to 04 in order
open http://localhost:5000
```

Script	What it does
01_start_cluster.sh	docker-compose up -d, then waits until hdfs dfsadmin -report succeeds
02_upload_data.sh	Generates the sample CSV if missing, creates /covid19/input and /covid19/output, uploads with hdfs dfs -put -f
03_run_mapreduce.sh	Clears old output, submits the streaming job to YARN, exports the result to results/mapreduce_output.csv
04_run_spark.sh	spark-submit to spark://spark-master:7077 in client mode, 1 GB and 2 cores per executor, 4 shuffle partitions

6.2 Local mode

```
pip3 install pyspark flask pandas
bash scripts/05_run_local.sh   # generate data, Spark, MapReduce via pipes, dashboard
```

Local mode replaces the cluster MapReduce with a Unix pipe, `mapper | sort | reducer`, which is exactly what Hadoop Streaming does, minus the distribution. PySpark 4.x needs Java 17 or 21 on the host.

6.3 Web interfaces

Interface	URL
Dashboard	http://localhost:5000
HDFS NameNode	http://localhost:9870
YARN ResourceManager	http://localhost:8088
Spark master	http://localhost:8080

7. Results

On the generated dataset the pipeline reports 2,202,263,683 cases and 34,053,607 deaths across 25 countries, which gives a global case-fatality rate of 1.55%. The smoothed global curve peaks on 2023-06-22 at about 2,506,511 new cases a day, and shows four waves, one per year, as expected from the generator's 60-day sine period stretched over the date range.

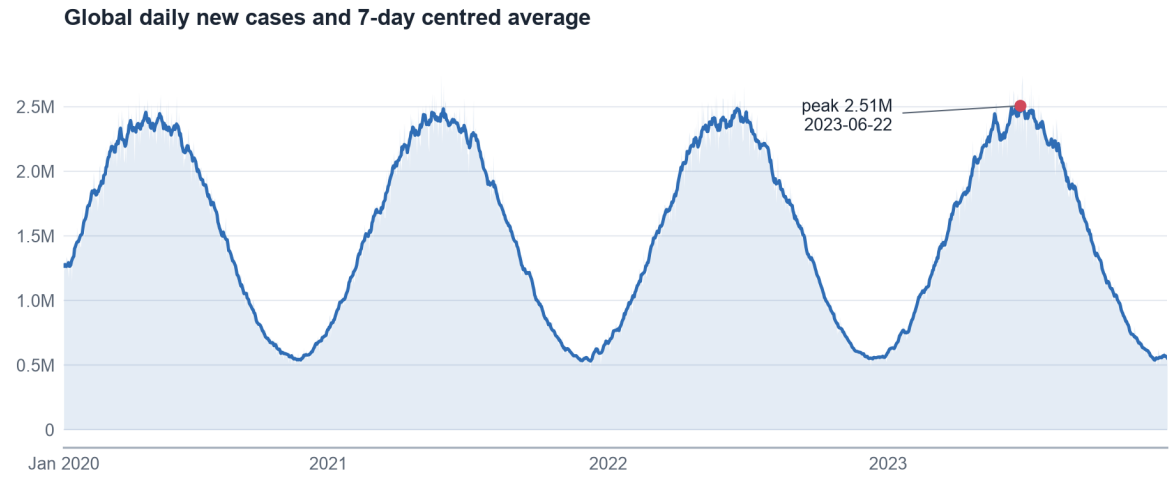


Figure 8. Global daily cases (shaded) and the 7-day average from Spark Job 3

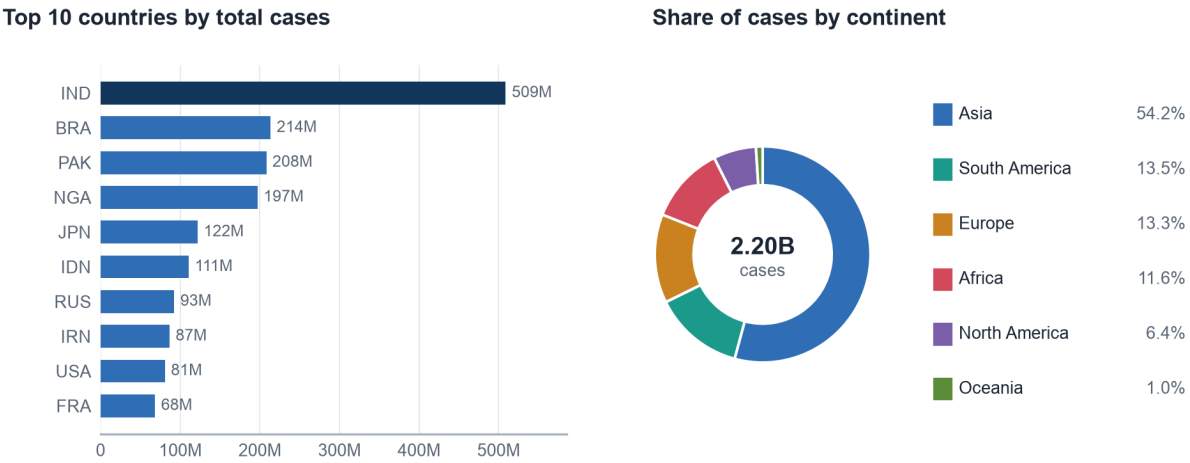


Figure 9. Top 10 countries (Job 1) and share of cases by continent (Job 2)

Top five countries

#	Country	Continent	Total cases	Total deaths	CFR
1	IND	Asia	509,003,126	7,957,872	1.56%
2	BRA	South America	213,586,379	2,139,162	1.00%
3	PAK	Asia	208,262,146	2,471,097	1.19%
4	NGA	Africa	197,088,368	4,310,247	2.19%
5	JPN	Asia	121,863,756	869,047	0.71%

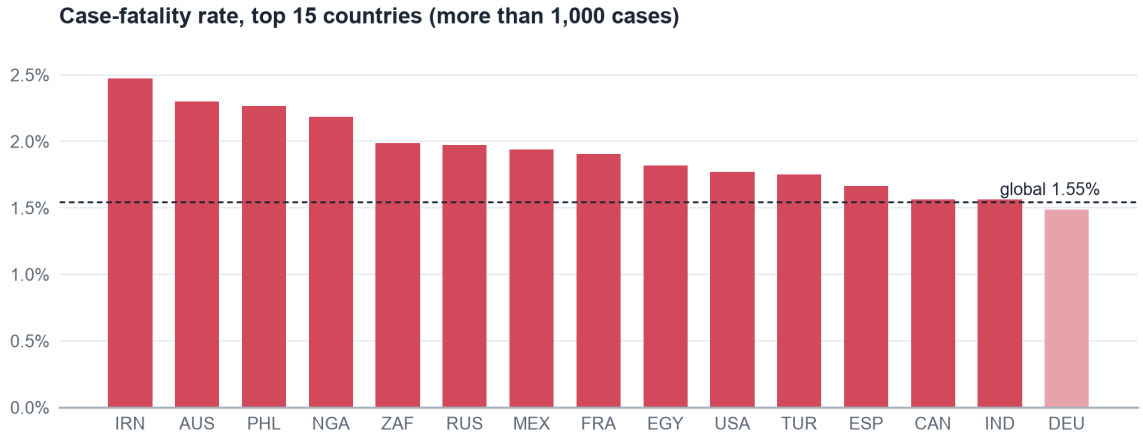


Figure 10. Case-fatality rate by country (Job 4); darker bars are above the global rate

The highest case-fatality rate is IRN at 2.475%, and the highest vaccination share is DEU at 69.6%. The vaccination figures cluster around 69% because the generator caps doses at twice the population and counts 60% of doses as people vaccinated.

8. Testing and Verification

Each part was run and checked on its own, then the whole cluster was torn down, volumes included, and rebuilt from nothing with `run_all.sh`.

Check	How	Result
Spark jobs 1–5	Recomputed every output independently with pandas and compared row by row	All match
MapReduce (local)	mapper | sort | reducer compared with pandas group-by sums	All 25 countries match
MapReduce (YARN)	Cluster output compared with the local pipe output using diff	Identical
Split handling	Fed the mapper two halves of the file separately, with the header in COVID_HEADER	Identical to single pass
MapReduce vs Spark	Sum of new_cases (MapReduce) vs max of total_cases (Spark Job 1)	Equal for every top-10 country
HDFS	hdfs dfsadmin -report and hdfs dfs -ls	2 live DataNodes, file stored with replication 2
YARN	ResourceManager REST API, job counters	2 NodeManagers RUNNING; 2 map tasks, 1 reduce task
Spark cluster	Spark master JSON API	2 workers ALIVE; application FINISHED with 4 cores
Dashboard	HTTP requests to every route; all six views rendered in headless Chrome in light, dark and narrow layouts; empty results folder	All routes return 200, no console errors, empty state shown

9. Limitations and Future Work

- **Synthetic data.** The bundled numbers are generated, not observed. Running on the real OWID file needs no code changes, but the reports should then be re-checked.
- **Spark reads from a shared folder, not HDFS.** Pointing `COVID_DATA` at `hdfs://namenode:9000/covid19/input/...` would make Spark use HDFS too.
- **Single-partition window.** Job 3 sorts all dates in one partition; fine for 1,440 rows, not for millions.
- **Old base images.** The Hadoop images are from 2019 (Debian 9, Java 8, Python 3.5) and the Spark image comes from Bitnami's legacy archive. Both work today but will not receive updates.
- **Single machine.** All containers share one host, so the cluster shows the architecture but not real scaling or fault tolerance. Running the same images on several machines, or on Kubernetes, would be the next step.
- **Results are read from files.** The dashboard shows whatever is in results/; it does not trigger jobs itself. A button that submits the pipeline would make it a control panel as well.